

AMD GPU

제조사별 GPU 차이 NVIDIA GPU (Hopper/Blackwell)

- **사실상 업계 표준** — 가장 성숙한 소프트웨어 생태계 (CUDA, cuDNN, TensorRT)
- 모든 ML 프레임워크가 NVIDIA 최우선 지원
- NVLink/NVSwitch로 멀티 GPU 스케일링 최강
- Transformer Engine (FP8) 등 AI 전용 하드웨어 가속 내장
- **단점:** 가격이 가장 비쌌, 공급 부족, 벤더 락인(CUDA)

AMD GPU (Instinct MI series)

- **범용성** — 그래픽, HPC, AI 모두 가능
- ROCm 소프트웨어 스택 (CUDA 대안)
- PyTorch/JAX 지원하지만 NVIDIA 대비 생태계가 아직 미성숙
- MI300X 등 최신 제품은 HBM 대용량으로 LLM 추론에서 경쟁력 확보 중

AWS Trainium

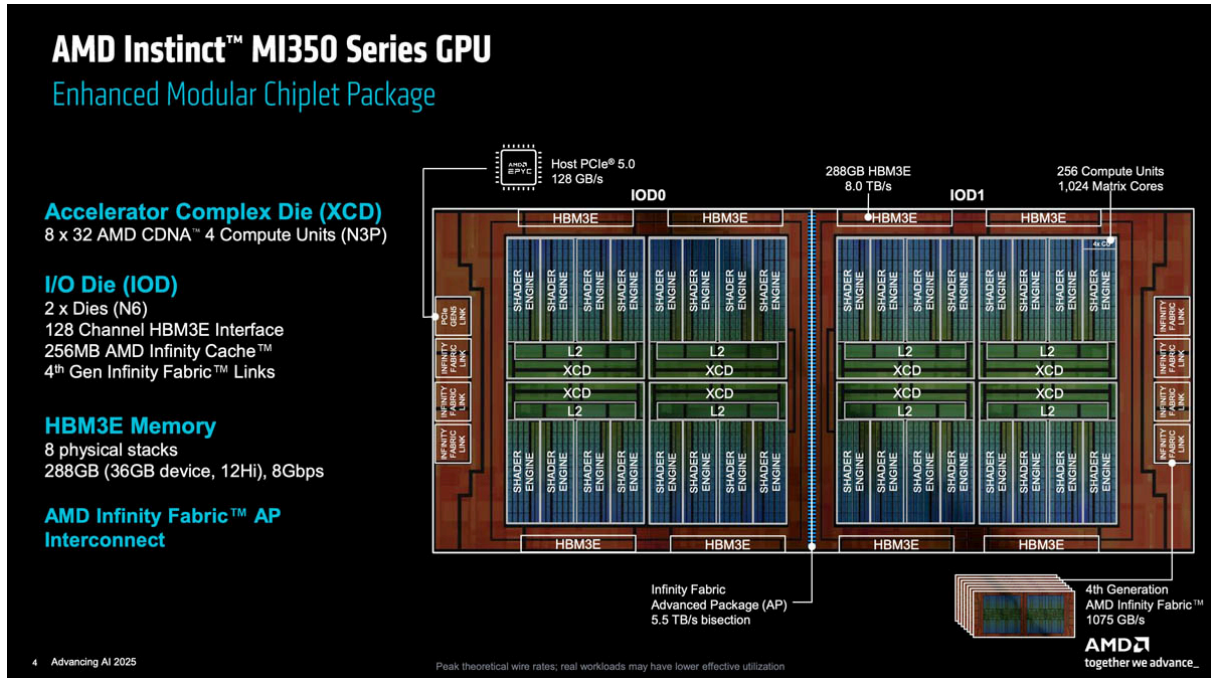
- AWS가 설계한 **학습 특화** 커스텀 칩 (Trainium2 세대 진행 중)
- EC2 `trn1` 인스턴스로만 사용 가능 (AWS 종속)
- Neuron SDK를 통해 PyTorch/JAX 모델 컴파일
- **가격 대비 성능** — NVIDIA GPU 인스턴스 대비 최대 50% 비용 절감을 목표
- 추론용은 별도로 **Inferentia** 칩 라인업이 있음

Furiosa AI NPU (RNGD 등)

- 한국 스타트업이 설계한 **추론 전용** 프로세서
- 데이터센터 추론 워크로드 타겟 (특히 Transformer 계열)
- 낮은 전력 소비, 높은 추론 처리량이 강점
- 자체 SDK/컴파일러 제공

- 아직 클라우드 대규모 가용성은 제한적 (자체 서비스 및 파트너 제공)

AMD MI series



1. Accelerator Complex Die (XCD) — 연산 엔진

- 구성: 8개 XCD, 각각 32개 CDNA 4 Compute Unit 탑재 → 총 **256 CU, 1,024 Matrix Cores**
- 역할: 실제 행렬 연산(GEMM), 벡터 연산을 수행하는 **핵심 연산 유닛**
- 각 XCD 안에는:
 - **Shader Engine** — 범용 셰이더/컴퓨터 연산 처리
 - **Matrix Core** — FP8/FP16/BF16 등 AI 행렬곱 전용 가속기
 - **L2 Cache** — XCD 내부 데이터 재사용을 위한 로컬 캐시

비유: CPU의 코어에 해당. 실제 숫자를 계산하는 "일꾼"들

2. I/O Die (IOD) — 통신 허브

- 구성: 2개 다이 (IOD0, IOD1), TSMC N6 공정

- **역할:** XCD들과 외부 세계를 연결하는 **중앙 스위치/라우터**
- 포함 기능:
 - **128채널 HBM3E 메모리 컨트롤러** — 메모리 접근 관리
 - **256MB Infinity Cache** — HBM 접근 전 히트율을 높여 대역폭 절약
 - **4세대 Infinity Fabric Links** — XCD 간, 다른 GPU 간 데이터 전송
 - **PCIe 5.0 인터페이스** — 호스트 CPU(EPYC)와 연결 (128 GB/s)

비유: 공장의 물류 센터. 원자재(데이터)를 작업장(XCD)에 분배하고, 외부와 소통

3. HBM3E Memory — 대용량 고대역폭 메모리

- **역할:** 모델 가중치, 활성화값, KV 캐시 등을 저장하는 **주 메모리**

비유: 작업대 위의 거대한 책상. 넓을수록(용량) 큰 모델을 올릴 수 있고, 빠를수록(대역폭) 연산 유닛이 굶지 않음

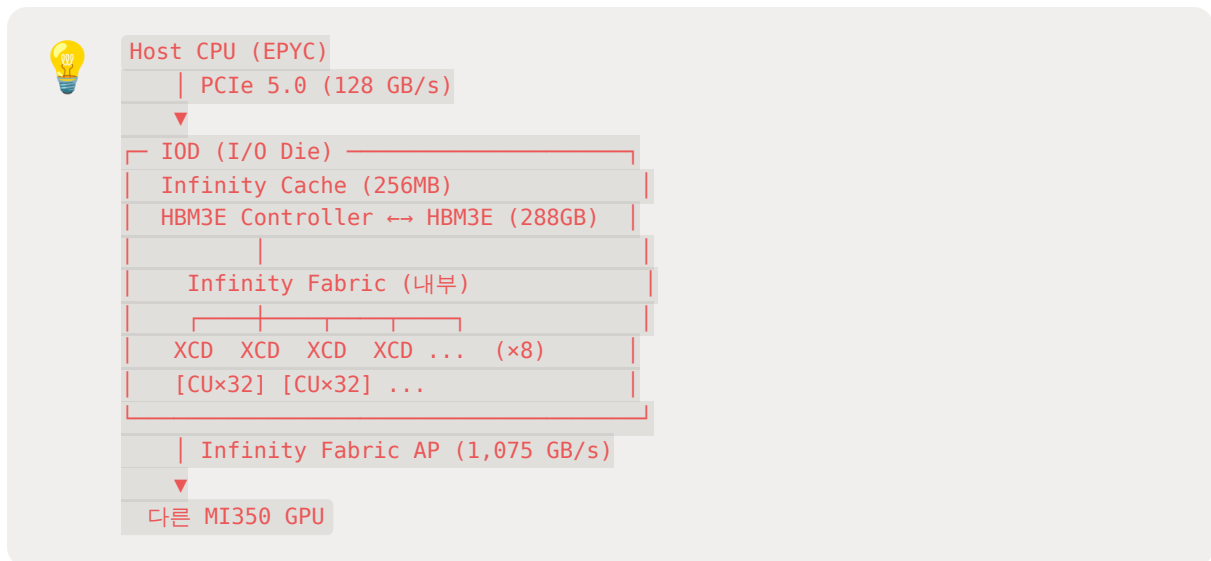
4. Infinity Fabric AP Interconnect — GPU 간 연결

- **역할:** 멀티 GPU 스케일링 시 GPU끼리 직접 데이터를 주고받는 **고속 인터커넥트**

비유: NVIDIA의 NVLink에 대응. 여러 GPU가 하나처럼 협력할 수 있게 해주는 고속도로

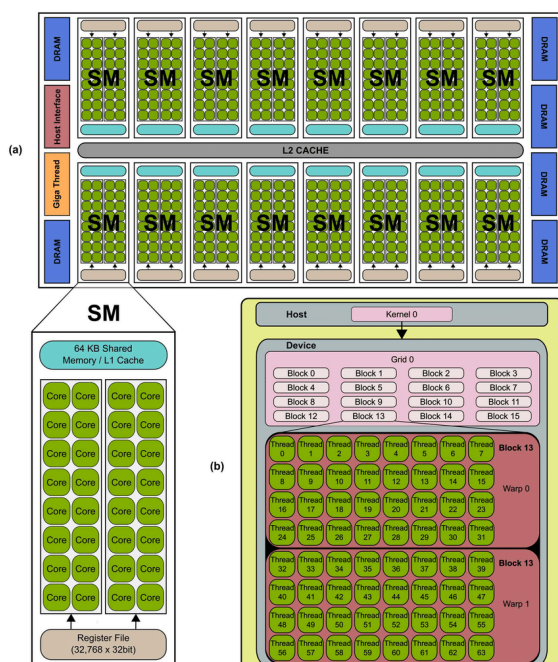
AMD (MI350)	NVIDIA (H100/B200)	역할
XCD (Accelerator Complex Die)	GPC (Graphics Processing Cluster)	CU/SM 여러 개를 묶은 연산 클러스터 단위
Shader Engine (CU 내부)	SM (Streaming Multiprocessor)	범용 연산을 수행하는 기본 실행 유닛
Matrix Core	Tensor Core	FP8/FP16/BF16 행렬곱 전용 가속기
L2 Cache (XCD 내)	L2 Cache (GPC/SM 공유)	온칩 데이터 재사용 캐시
IOD (I/O Die)	GigaThread Engine + Memory Controller	메모리 컨트롤러, PCIe 인터페이스, 전체 스케줄링을 담당하는 중앙 허브
HBM3E	HBM3/HBM3E	고대역폭 주 메모리 (동일)
Infinity Fabric AP	NVLink	GPU 간 고속 직접 연결 인터커넥트

데이터 흐름 요약

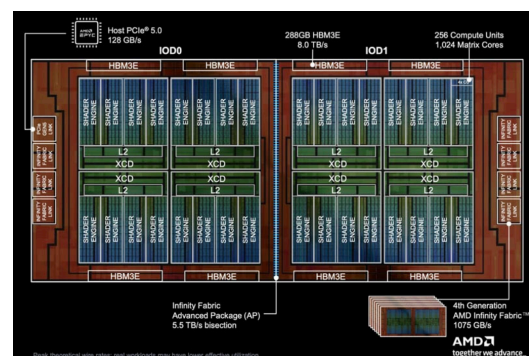


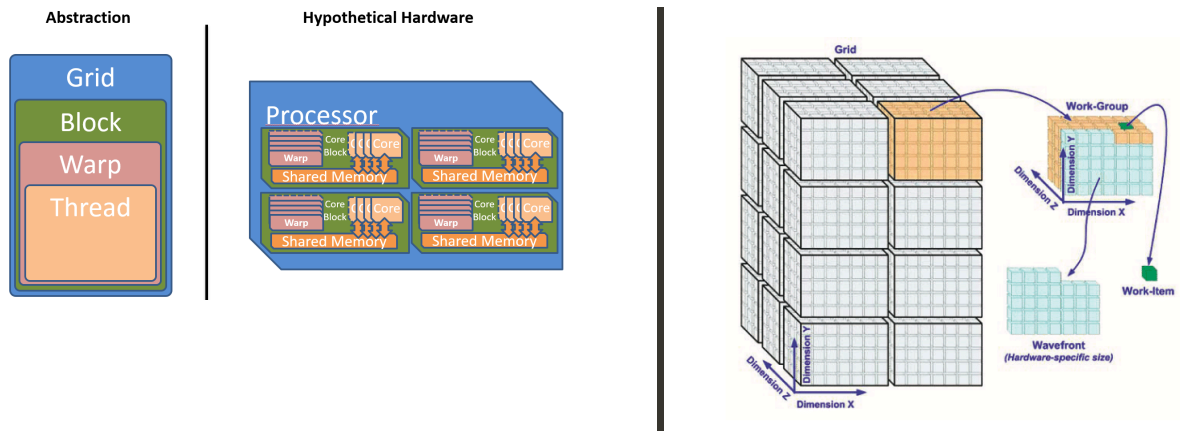
Nvidia와 실행 계층 비교

NVIDIA



AMD





NVIDIA	AMD (CDNA/RDNA)	역할
Grid	Grid (NDRange in OpenCL)	전체 작업 공간
Block (Thread Block)	Workgroup	하나의 CU에 할당되는 스레드 묶음
Warp (32 threads)	Wavefront (64 threads*)	SIMD** 실행 단위 (lockstep)
Thread	Work-item	개별 실행 단위

** **SIMD**(Single Instruction Multiple Data): 하나의 명령어로 여러 개의 데이터를 동시에 처리하는 병렬 처리 기법

• 하드웨어 매핑

NVIDIA	AMD
SM (Streaming Multiprocessor)	CU (Compute Unit)
SM 안에 4개 Warp Scheduler	CU 안에 4개 SIMD Unit
각 Scheduler가 Warp 1개 발행	각 SIMD가 Wavefront 1개 실행

⇒ NVIDIA SM은 내부적으로 **4개의 독립된 파티션으로 나뉘어 있고, 각 파티션이 Warp Scheduler와 실행 유닛 세트를 가짐**. 매 사이클마다 각 스케줄러가 ready 상태인 warp를 하나씩 골라 동시에 발행. 즉 한 사이클에 최대 **4개 warp의 명령어가 병렬로 실행 유닛에 들어감**, 이를 통해 실행 유닛 가동률을 높이고 메모리 지연을 숨김.



SM

- └ Partition 0: Warp Scheduler 0 → FP32, INT32, Tensor Core, LD/ST, SFU
- └ Partition 1: Warp Scheduler 1 → FP32, INT32, Tensor Core, LD/ST, SFU
- └ Partition 2: Warp Scheduler 2 → FP32, INT32, Tensor Core, LD/ST, SFU
- └ Partition 3: Warp Scheduler 3 → FP32, INT32, Tensor Core, LD/ST, SFU

⇒ AMD **CU**에는 스케줄러가 하나뿐이며, 매 사이클마다 4개의 SIMD 유닛 중 하나를 라운드로빈으로 돌아가며 wavefront를 발행. 대신 **wavefront 자체가 64-thread로 넓고** CU 수를 많이 배치해서, 스케줄러가 단순한 대신 전체 처리량을 확보하는 방식.



```
CU (Compute Unit) - CDNA 3/4
├─ Scheduler (1개, 라운드로빈)
│   ├── SIMD0 (16-wide ALU) + MFMA Unit (Matrix FMA)
│   ├── SIMD1 (16-wide ALU) + MFMA Unit
│   ├── SIMD2 (16-wide ALU) + MFMA Unit
│   └── SIMD3 (16-wide ALU) + MFMA Unit
├─ Scalar Unit
├─ LD/ST Unit
├─ LDS (64KB)
└─ L1 Cache
```